the communicating endpoints. With this reasoning, the only type of failure that destroys communication is one that also destroys one or more of the endpoints, which obviously destroys the overall communication anyhow. Fate sharing is one of the design philosophies that allows virtual connections (e.g., those implemented by TCP) to remain active even if connectivity within the network has failed for a (modest) period of time. Fate sharing also supports a "dumb network with smart end hosts" model, and one of the ongoing tensions in today's Internet is what functions reside in the network and what functions do not.

1.1.3 Error Control and Flow Control

There are some circumstances where data within a network gets damaged or lost. This can be for a variety of reasons such as hardware problems, radiation that modifies bits while being transmitted, being out of range in a wireless network, and other factors. Dealing with such errors is called *error control*, and it can be implemented in the systems constituting the network infrastructure, or in the systems that attach to the network, or some combination. Naturally, the end-to-end argument and fate sharing would suggest that error control be implemented close to or within applications.

Usually, if a small number of bit errors are of concern, a number of mathematical codes can be used to detect and repair the bit errors when data is received or while it is in transit [LC04]. This task is routinely performed within the network. When more severe damage occurs in a packet network, entire packets are usually resent or *retransmitted*. In circuit-switched or VC-switched networks such as X.25, retransmission tends to be done inside the network. This may work well for applications that require strict in-order, error-free delivery of their data, but some applications do not require this capability and do not wish to pay the costs (such as connection establishment and potential retransmission delays) to have their data reliably delivered. Even a reliable file transfer application does not really care in what order the chunks of file data are delivered, provided it is eventually satisfied that all chunks are delivered without errors and can be reassembled back into the original order.

As an alternative to the overhead of reliable, in-order delivery implemented within the network, a different type of service called *best-effort delivery* was adopted by Frame Relay and the Internet Protocol. With best-effort delivery, the network does not expend much effort to ensure that data is delivered without errors or gaps. Certain types of errors are usually detected using error-detecting codes or *checksums*, such as those that might affect where a datagram is directed, but when such errors are detected, the errant datagram is merely discarded without further action.

If best-effort delivery is successful, a fast sender can produce information at a rate that exceeds the receiver's ability to consume it. In best-effort IP networks, slowing down a sender is achieved by *flow control* mechanisms that operate outside the network and at higher levels of the communication system. In particular,

TCP handles this type of problem, and we shall discuss it in detail in Chapters 15 and 16. This is consistent with the end-to-end argument: TCP, which resides at the end hosts, handles rate control. It is also consistent with fate sharing: the approach allows some elements of the network infrastructure to fail without necessarily affecting the ability of the devices outside the network to communicate (as long as some communication path continues to operate).

1.2 Design and Implementation

Although a protocol architecture may suggest a certain approach to implementation, it usually does not include a mandate. Consequently, we make a distinction between the protocol architecture and the *implementation architecture*, which defines how the concepts in a protocol architecture may be rendered into existence, usually in the form of software.

Many of the individuals responsible for implementing the protocols for the ARPANET were familiar with the software structuring of operating systems, and an influential paper describing the "THE" multiprogramming system [D68] advocated the use of a hierarchical structure as a way to deal with verification of the logical soundness and correctness of a large software implementation. Ultimately, this contributed to a design philosophy for networking protocols involving multiple *layers* of implementation (and design). This approach is now called *layering* and is the usual approach to implementing protocol suites.

1.2.1 Layering

With layering, each layer is responsible for a different facet of the communications. Layers are beneficial because a layered design allows developers to evolve different portions of the system separately, often by different people with somewhat different areas of expertise. The most frequently mentioned concept of protocol layering is based on a standard called the *Open Systems Interconnection* (OSI) model [Z80] as defined by the International Organization for Standardization (ISO). Figure 1-2 shows the standard OSI layers, including their names, numbers, and a few examples. The Internet's layering model is somewhat simpler, as we shall see in Section 1.3.

Although the OSI model suggests that seven logical layers may be desirable for modularity of a protocol architecture implementation, the TCP/IP architecture is normally considered to consist of five. There was much debate about the relative benefits and deficiencies of the OSI model, and the ARPANET model that preceded it, during the early 1970s. Although it may be fair to say that TCP/IP ultimately "won," a number of ideas and even entire protocols from the ISO protocol suite (protocols standardized by ISO that follow the OSI model) have been adopted for use with TCP/IP (e.g., IS-IS [RFC3787]).

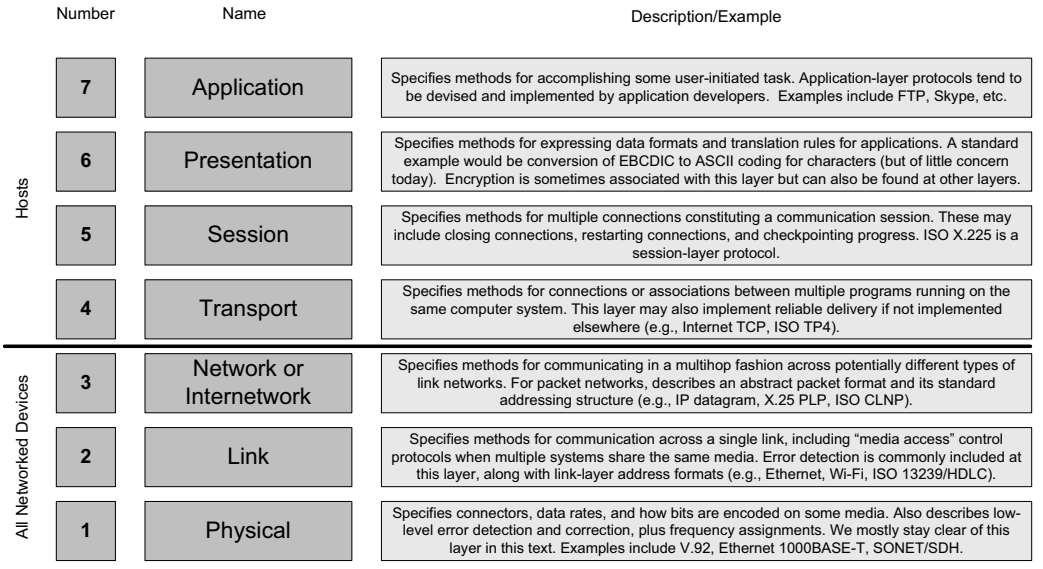

Figure 1-2 The standard seven-layer OSI model as specified by the ISO. Not all protocols are implemented by every networked device (at least in theory). The OSI terminology and layer numbers are widely used.

As described briefly in Figure 1-2, each layer has a different responsibility. From the bottom up, the *physical* layer defines methods for moving digital information across a communication medium such as a phone line or fiber-optic cable. Portions of the Ethernet and Wireless LAN (Wi-Fi) standards are here, although we do not delve into this layer very much in this text. The *link* or *data-link* layer includes those protocols and methods for establishing connectivity to a neighbor sharing the same medium. Some link-layer networks (e.g., DSL) connect only two neighbors. When more than one neighbor can access the same shared network, the network is said to be a *multi-access* network. Wi-Fi and Ethernet are examples of such multi-access link-layer networks, and specific protocols are used to mediate which stations have access to the shared medium at any given time. We discuss these in Chapter 3.

Moving up the layer stack, the *network* or *internetwork* layer is of great interest to us. For packet networks such as TCP/IP, it provides an interoperable packet format that can use different types of link-layer networks for connectivity. The layer also includes an addressing scheme for hosts and routing algorithms that choose where packets go when sent from one machine to another. Above layer 3 we find protocols that are (at least in theory) implemented only by end hosts, including the *transport* layer. Also of great interest to us, it provides a flow of data between sessions and can be quite complex, depending on the types of services it provides

(e.g., reliable delivery on a packet network that might drop data). *Sessions* represent ongoing interactions between applications (e.g., when "cookies" are used with a Web browser during a Web login session), and session-layer protocols may provide capabilities such as connection initiation and restart, plus *checkpointing* (saving work that has been accomplished so far). Above the session layer we find the *presentation* layer, which is responsible for format conversions and standard encodings for information. As we shall see, the Internet protocols do not include a formal session or presentation protocol layer, so these functions are implemented by applications if needed.

The top layer is the *application* layer. Applications usually implement their own application-layer protocols, and these are the ones most visible to users. There is a wide variety of application-layer protocols, and programmers are constantly inventing new ones. Consequently, the application layer is where there is the greatest amount of innovation and where new capabilities are developed and deployed.

1.2.2 Multiplexing, Demultiplexing, and Encapsulation in Layered Implementations

One of the major benefits of a layered architecture is its natural ability to perform *protocol multiplexing*. This form of multiplexing allows multiple different protocols to coexist on the same infrastructure. It also allows multiple instantiations of the same protocol object (e.g., connections) to be used simultaneously without being confused.

Multiplexing can occur at different layers, and at each layer a different sort of *identifier* is used for determining which protocol or stream of information belongs together. For example, at the link layer, most link technologies (such as Ethernet and Wi-Fi) include a *protocol identifier* field value in each packet to indicate which protocol is being carried in the link-layer frame (IP is one such protocol). When an object (packet, message, etc.), called a *protocol data unit* (PDU), at one layer is carried by a lower layer, it is said to be *encapsulated* (as opaque data) by the next layer down. Thus, multiple objects at layer N can be multiplexed together using encapsulation in layer N - 1. Figure 1-3 shows how this works. The identifier at layer N - 1 is used to determine the correct receiving protocol or program at layer N during demultiplexing.

In Figure 1-3, each layer has its own concept of a message object (a PDU) corresponding to the particular layer responsible for creating it. For example, if a layer 4 (transport) protocol produces a packet, it would properly be called a layer 4 PDU or *transport PDU* (TPDU). When a layer is provided a PDU from the layer above it, it usually "promises" to not look into the contents of the PDU. This is the essence of encapsulation—each layer treats the data from above as opaque, uninterpretable information. Most commonly a layer prepends the PDU with its own header, although trailers are used by some protocols (not TCP/IP). The header is used for multiplexing data when sending, and for the receiver to perform demultiplexing,

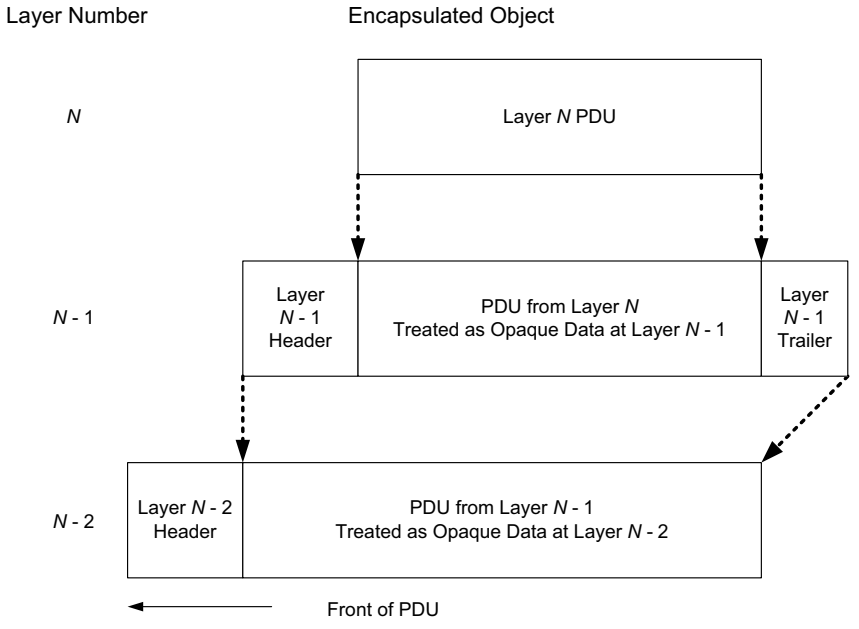

Figure 1-3 Encapsulation is usually used in conjunction with layering. Pure encapsulation involves taking the PDU of one layer and treating it as opaque (uninterpreted) data at the layer below. Encapsulation takes place at each sender, and decapsulation (the reverse operation) takes place at each receiver. Most protocols use headers during encapsulation; a few also use trailers.

based on a demultiplexing (demux) identifier. In TCP/IP networks such identifiers are commonly hardware addresses, IP addresses, and port numbers. The header may also include important state information, such as whether a virtual circuit is being set up or has already completed setup. The resulting object is another PDU.

One other important feature of layering suggested by Figure 1-2 is that in pure layering not all networked devices need to implement all the layers. Figure 1-4 shows that in some cases a device needs to implement only a few layers if it is expected to perform only certain types of processing.

In Figure 1-4, a somewhat idealized small internet includes two end systems, a switch, and a router. In this figure, each number corresponds to a type of protocol at a particular layer. As we can see, each device implements a different subset of the layer stack. The host on the left implements three different link-layer protocols (D, E, and F) with corresponding physical layers and three different transport-layer protocols (A, B, and C) that run on a single type of network-layer protocol. End hosts implement all the layers, switches implement up to layer 2 (this switch implements D and G), and routers implement up to layer 3. Routers are capable of interconnecting different types of link-layer networks and must implement the link-layer protocols for each of the network types they interconnect.